

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/TQE.2020.DOI

A Differentiable Ising Module for Hybrid Quantum-Classical Neural Networks

MATTEO MARCHIORI¹, EMILIANO TOLOTTI¹, DANIELE FRANCH², ENRICO BLANZIERI^{1,3},
and DAVIDE PASTORELLO^{4,3}

¹Department of Information Engineering and Computer Science - University of Trento, Trento, Italy

²University of Iceland, 102 Reykjavik, Iceland

³INFN-TIFPA, Trento Institute for Fundamental Physics and Applications, Trento, Italy

⁴Department of Mathematics - University of Bologna, 40126 Bologna, Italy

Corresponding author: Matteo Marchiori (email: matteo.marchiori03@gmail.com)

The authors gratefully acknowledge CINECA for providing computing time on the D-Wave quantum annealer.

ABSTRACT Ising machines and quantum annealers solve combinatorial optimization problems by searching for low-energy spin configurations. Recent work has shown that Ising energy minimization can itself serve as a trainable supervised model. We turn this model into a reusable deep learning component: a differentiable Ising module. Its forward pass calls an annealing solver to compute the ground-state energy of a parametric Ising Hamiltonian; its backward pass computes the analytic gradient with respect to the couplings from the spin configuration the solver returns. The module is therefore compatible with automatic differentiation in PyTorch. On top of it we build the *Ising network*, a shallow architecture that combines several Ising modules through a trainable linear readout. We compare the single module against the network on nine UCI benchmarks, both in their original imbalanced form and in a balanced variant, and on XOR problems of growing dimensionality. The network matches or outperforms the single module on every benchmark: it removes the tendency to collapse onto one class under imbalance and captures interactions up to six dimensions, while remaining trainable within the standard PyTorch ecosystem. A final experiment trains both models on a D-Wave quantum processor and reproduces the simulated results on a reduced XOR task.

INDEX TERMS Energy-based models, hybrid quantum-classical neural networks, Ising machines, Ising model, quantum annealing, quantum machine learning.

I. INTRODUCTION

THE Ising model originated as a description of interacting spins in statistical mechanics [1]–[3], but it applies well beyond magnetism: a large class of combinatorial optimization problems can be written as the search for the lowest-energy configuration of a spin system, and many NP-hard problems admit a direct Ising formulation [4]–[6]. This observation motivates the construction of Ising machines, devices and algorithms built specifically to minimize an Ising energy [7], [8]. A problem is encoded in the local fields and pairwise couplings of the spins, and the machine returns a low-energy configuration as a candidate solution. Implementations range from simulated annealing [9] to dedicated physical hardware.

Quantum annealing is a general paradigm for solving such problems on quantum hardware. The idea, introduced for the transverse-field Ising model by [10], is to evolve a quantum

system slowly from the ground state of a simple Hamiltonian toward that of the problem Hamiltonian, so that the measured spin configuration approximates the optimum [11]–[13]. Quantum annealers are the hardware realization of this paradigm; the D-Wave Ocean SDK [14] exposes one such device through a software interface, and embedding an all-to-all problem onto the sparse qubit graph carries an additional overhead. Annealing-based solvers have been applied across scheduling, routing, sampling and constraint satisfaction, which makes the Ising minimization a reusable primitive rather than a problem-specific technique.

In parallel, machine learning has become a natural target for these solvers. Classical neural networks remain the dominant paradigm for supervised learning [15], and a substantial body of work asks how quantum resources can play a comparable role. One direction trains parameterized quantum circuits as differentiable models [16]–[18], optimized by

gradient descent through the parameter-shift rule [19]–[21]; these range from circuit-based quantum neural networks to quantum perceptrons realized on actual hardware [22], and to hybrid classical-quantum models that interleave quantum layers with ordinary neural networks [23]. A second direction uses annealers directly, either to train classifiers [24], [25] or, closer to our setting, to sample from energy-based models in the spirit of Boltzmann machines [26]. In both cases, the learning scheme must be realizable on present-day noisy intermediate-scale quantum hardware.

Schmid, Zardini and Pastorello [27] took a more direct route: they showed that the Ising energy minimization can itself serve as a parametric supervised model, with the data entering as local fields, the couplings acting as trainable weights, and the ground-state energy as the output. The model is trained by gradient descent, and the gradients are estimated from the spin configurations returned by the machine rather than computed by explicit backpropagation. A useful consequence is that the systematic error of an approximate solver is partly absorbed during training. What that work leaves open is how to use such a model as an ordinary building block inside the deep learning ecosystem, where layers are composed freely and trained by automatic differentiation.

This paper addresses that gap. We recast the parametric Ising model as a differentiable module that exposes a forward and a backward interface and plugs directly into PyTorch. The forward pass calls an annealing solver to estimate the ground-state energy of the spin system; the backward pass propagates gradients to the couplings through the analytic expression of Schmid *et al.*, evaluated on the stored spin configurations. Because the module behaves like any other layer, it inherits the surrounding toolchain: optimizers, loss functions, learning-rate schedulers, mini-batching and compatibility with GPU tensors. On top of it we build the *Ising network*, a shallow architecture in which several Ising modules run in parallel and are combined by a trainable linear readout.

The contribution of this work is threefold. First, we introduce a differentiable Ising energy module that carries the analytic coupling gradient into automatic differentiation, turning an Ising solver into a standard trainable layer. Second, we propose the Ising network, a hybrid architecture that assembles a set of energy-based units into a single trainable layer through a linear readout. Third, we report an empirical study on nine UCI benchmarks, evaluated both in their native class-imbalanced form and in a pre-balanced variant, and on a family of synthetic XOR problems of growing dimensionality, comparing the single module against the network, together with a first run of both models on a D-Wave quantum processor.

The remainder of the paper is organized as follows. Section II reviews the background: the Ising model, its realization through quantum annealing, and the parametric learning model of Schmid *et al.*, including its analytic gradients and the offset scheme for hidden spins. Section III presents our

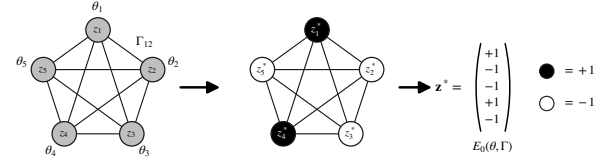


FIGURE 1. On the left, an Ising model on a fully connected graph with $|V| = 5$ spins z , biases θ and couplings Γ . On the right, the output of an Ising machine: a $\{-1, +1\}$ assignment (white/black nodes) for each spin, that is, the configuration z^* together with its minimal energy $E_0(\theta, \Gamma)$. Adapted from [27].

proposal, namely the differentiable Ising module and the Ising network built on top of it. Section IV describes the experimental setup and reports the results on the UCI and XOR benchmarks, including a run on quantum hardware. Section V discusses these findings and Section VI concludes.

II. BACKGROUND

This section collects the background our method builds on: the Ising model, its realization through quantum annealing, and the parametric learning model of Schmid *et al.* [27] on which the differentiable module rests.

A. THE ISING MODEL

The Ising model was introduced to describe ferromagnetic systems [1] and has since become a standard framework in statistical mechanics. It is defined on a graph (V, E) , where each vertex $i \in V$ carries a discrete spin variable $z_i \in \{-1, +1\}$. Each spin has a local bias $\theta_i \in \mathbb{R}$, and each edge $(i, j) \in E$ a pairwise coupling $\Gamma_{ij} \in \mathbb{R}$. The total energy of a configuration is given by the Hamiltonian:

$$E(\theta, \Gamma, z) = \sum_{i \in V} \theta_i z_i + \sum_{(i, j) \in E} \Gamma_{ij} z_i z_j. \quad (1)$$

An Ising machine seeks the spin configuration that minimizes this Hamiltonian:

$$z^* = \arg \min_z E(\theta, \Gamma, z), \quad (2)$$

where z^* is the ground-state configuration. Because many discrete problems admit an Ising formulation [4], this minimization encodes a wide class of combinatorial optimization tasks, and the devices built to perform it, known as Ising machines, range from digital simulated annealing [9] to coherent optical machines and quantum annealers [7].

B. QUANTUM ANNEALING

Quantum annealing solves optimization problems by evolving a quantum system toward the ground state of a problem Hamiltonian [10]. The Ising minimization of Eq. 1 maps to the ground-state search of a corresponding Hamiltonian. Consider the time-dependent Hamiltonian

$$H(t) = \gamma(t)H_D + H_P \quad (3)$$

where H_P is the problem Hamiltonian, H_D is the transverse-field Hamiltonian, and $\gamma : \mathbb{R}^+ \rightarrow \mathbb{R}$ is a decreasing function that gradually suppresses the kinetic term H_D , guiding the system toward the ground state of H_P . The transverse field provides the quantum fluctuations that let the system explore the solution landscape.

Quantum annealing is realized physically on a network of qubits placed on the vertices of a graph (V, E) , with $|V| = n$, whose edges E carry the couplings between qubits. The problem Hamiltonian is a self-adjoint operator on the n -qubit Hilbert space $(\mathbb{C}^2)^{\otimes n}$:

$$H_P = \sum_{i \in V} \theta_i \sigma_z^{(i)} + \sum_{(i,j) \in E} \Gamma_{ij} \sigma_z^{(i)} \sigma_z^{(j)}, \quad (4)$$

where $\sigma_z^{(i)}$ is the Pauli- z operator acting on qubit i , and θ_i and Γ_{ij} are the local fields and the couplings between qubits. The transverse-field Hamiltonian is the sum of the Pauli- x operators on the same qubits,

$$H_D = - \sum_{i \in V} \sigma_x^{(i)}, \quad (5)$$

whose ground state is the uniform superposition of all the spin configurations. Quantum annealing is therefore a physical mechanism for minimizing the Ising Hamiltonian. The D-Wave processors implement this scheme on a sparse hardware graph, so mapping a fully connected problem requires an embedding step with an additional overhead [14].

C. PARAMETRIC MODEL

Schmid et al. [27] turned the Ising minimization into a supervised learning model. The model is defined as

$$F(\theta|\Gamma, \lambda, \varepsilon) = \lambda \min_{\mathbf{z} \in \{-1, 1\}^n} E(\theta, \Gamma, \mathbf{z}) + \varepsilon \quad (6)$$

$$= \lambda E_0(\theta, \Gamma) + \varepsilon. \quad (7)$$

The input data enter as the spin biases $\theta = (\theta_1, \dots, \theta_n)$, the couplings Γ_{ij} on the topology of the Ising machine are the trainable weights, and the output is the ground-state energy $E_0(\theta, \Gamma)$, rescaled by λ and shifted by ε . The two additional parameters let the model match the range of the target outputs, playing a role comparable to the weight and the bias of a linear layer. The arrangement is neural-like: spins play the role of neurons and couplings that of synaptic weights.

1) Training Phase

Given a training set $\mathcal{D} = \{(\theta^{(a)}, y^{(a)})\}_{a=1, \dots, N}$ with $y^{(a)} = f(\theta^{(a)})$, where $f : \mathcal{X} \rightarrow \mathbb{R}$ is the unknown target, the model (7) is trained by minimizing a loss function, for example the mean squared error (MSE),

$$L(\Gamma, \lambda, \varepsilon) = \frac{1}{N} \sum_{a=1}^N \left[F(\theta^{(a)}|\Gamma, \lambda, \varepsilon) - y^{(a)} \right]^2, \quad (8)$$

by gradient descent, updating each parameter against the gradient of L :

$$\delta\Gamma = -\eta \nabla_{\Gamma} L, \quad \delta\lambda = -\eta \frac{\partial L}{\partial \lambda}, \quad \delta\varepsilon = -\eta \frac{\partial L}{\partial \varepsilon}, \quad (9)$$

with learning rate $\eta > 0$.

The key observation of [27] is that these gradients need no explicit backpropagation through the minimization: they are read off the spin configuration z^* the machine already returns. Differentiating $F = \lambda E_0 + \varepsilon$ and using $E_0 = \sum_i \theta_i z_i^* + \sum_{(i,j)} \Gamma_{ij} z_i^* z_j^*$, one obtains the partial derivatives

$$\frac{\partial F}{\partial \Gamma_{ij}} = \lambda z_i^* z_j^*, \quad \frac{\partial F}{\partial \lambda} = E_0(\theta, \Gamma), \quad \frac{\partial F}{\partial \varepsilon} = 1. \quad (10)$$

In particular $\partial E_0 / \partial \Gamma_{ij} = z_i^* z_j^*$, so a single call to the machine yields both the prediction E_0 and the quantities needed to update the couplings. Schmid et al. prove that the derivatives of Eq. (10) are well defined almost everywhere in the parameter space [27]. Training therefore mirrors that of a neural network, with the conventional backpropagation step replaced by the Ising-machine evaluation of E_0 and z^* . A practical benefit is that exact ground states are not required: the systematic deviations of an approximate solver are absorbed by the parameter updates, which makes the scheme usable with real annealers.

2) Hidden Spins

On a complete topology the number of couplings Γ_{ij} grows quadratically with the input dimension n , so the model capacity is fixed by the dimensionality of the data, much like a neural network with only an input and an output layer. To add capacity, Schmid et al. introduce hidden spins: extra nodes in the topology graph that contribute additional couplings and hence additional parameters. They are obtained through a preprocessing map

$$h_{\text{pre}} : \mathbb{R}^n \rightarrow \mathbb{R}^{n_{\text{total}}}, \quad n_{\text{total}} = n + n_{\text{hidden}}, \quad (11)$$

that lifts the input θ from the feature space into a higher-dimensional space before it is fed to the Ising model. The map does not change the training procedure of Section II-C1, but its choice matters. Padding with zeros, for instance, makes the hidden spins indistinguishable and therefore redundant, while filling them with random values risks overshadowing the original input when $n_{\text{hidden}} \gg n$.

The scheme adopted in [27] is a constant real-valued offset initialization,

$$\theta \in \mathbb{R}^n \mapsto h_{\text{offset}}(\theta) = \begin{pmatrix} \theta \\ \theta + 1 \cdot \mathbf{d} \\ \vdots \\ \theta + (l-1) \cdot \mathbf{d} \end{pmatrix} \in \mathbb{R}^{n_{\text{total}}}, \quad (12)$$

with $\mathbf{d} \in \mathbb{R}^n$, $l \in \mathbb{Z}^+$ and $n_{\text{total}} = ln$. The input is concatenated with itself l times, each copy shifted by an increasing multiple of the offset $\mathbf{d}$, so that the hidden spins start from distinct biases yet stay tied to the original features.

In the reference implementation of [27] the offset $\mathbf{d}$ is a single scalar ε_h added to every component, and n_{total} need not be a multiple of n : the copies are tiled cyclically and the last one is truncated to match the annealer size. This is the hidden-spin scheme we reuse unchanged in the differentiable module of Section III.

III. DIFFERENTIABLE ISING MODULES AND NETWORKS

The parametric model of Section II-C is trained with the hand-coded update rules of Eq. (10), outside any deep-learning framework. We turn it into a reusable component. Our contribution is twofold: a differentiable Ising module that wraps the annealing call and the analytic coupling gradient into a single autograd primitive, and the Ising network, a shallow architecture that composes several such modules. The gradient formulas are those recalled in Section II-C1; what is new here is the construction that makes them interoperable with automatic differentiation and the architecture built on top.

A. THE ISING MODULE

1) Motivation

Automatic differentiation requires every operation in the computational graph to expose a backward rule. The ground-state map $\theta \mapsto E_0$ does not: it contains a minimization over the discrete set $\{-1, +1\}^n$, which is not differentiable and is, moreover, computed by an external solver that the framework cannot trace. The model of Schmid et al. sidesteps this through the analytic derivatives of Eq. (10), but those were applied by explicit update rules. The module below carries the same derivatives into a standard autograd primitive, so that an Ising solver can be dropped into a network and trained like any other layer.

2) Differentiable Ising Energy Function

We implement the ground-state energy as a custom autograd function with a forward and a backward rule. In the forward pass, for each input vector θ in the batch the annealer returns an approximate ground-state configuration z^* and the corresponding energy $E_0(\theta, \Gamma)$ of Eq. (7); the configuration z^* is cached for the backward pass. In the backward pass we use the analytic derivative of Eq. (10),

$$\frac{\partial E_0}{\partial \Gamma_{ij}} = z_i^* z_j^*,$$

evaluated on the cached z^* , and combine it with the incoming gradient $\partial L / \partial E_0$ by the chain rule. The backward rule also returns the envelope-theorem gradient with respect to the input biases, $\partial E_0 / \partial \theta_i = z_i^*$, so the function can be composed with upstream layers; in the experiments of this paper, however, θ carries the raw data and only the couplings Γ are trained through this function. No gradient flows through the annealer itself: the returned spin configurations are treated as constants. With both rules in place, the energy minimization

behaves as an ordinary node of a PyTorch graph. Algorithm 1 summarizes the two passes.

Algorithm 1 Differentiable Ising Energy Function (Forward and Backward)

- 1: **Input:** batch of biases $\{\theta_i\}$, coupling matrix Γ , annealer instance
 - 2: **Forward pass:**
 - 3: **for** each sample i in batch, in parallel across workers **do**
 - 4: Query the annealer with (θ_i, Γ) : obtain the approximate ground state z_i^* and its energy E_i
 - 5: Store z_i^* for backward
 - 6: **end for**
 - 7: **Return:** batch energies $\{E_i\}$
 - 8: **Backward pass:**
 - 9: **for** each sample i in batch **do**
 - 10: Compute outer product $z_i^* (z_i^*)^\top$
 - 11: Multiply by incoming gradient $\partial L / \partial E_i$
 - 12: **end for**
 - 13: Sum over batch and retain the strictly upper-triangular part to get $\partial L / \partial \Gamma$
 - 14: **Return:** $\partial L / \partial \Gamma, z^*$
-

3) The Ising Module

The Ising module wraps this function and adds the two learnable parameters of the parametric model, the scale λ and the offset ε , so that its output is

$$F(\theta) = \lambda E_0(\theta, \Gamma) + \varepsilon.$$

The scale and offset let the module match the range of the target outputs, playing a role comparable to the weight and bias of a linear layer. All three quantities Γ , λ and ε are registered as parameters and updated by the surrounding optimizer, so the module trains end-to-end. Algorithm 2 summarizes the forward pass.

Algorithm 2 Forward Pass of the Ising Module

- 1: Compute E_0 using the Ising Energy Function
 - 2: Scale and shift: $F = \lambda \cdot E_0 + \varepsilon$
 - 3: **Return:** F
-

4) Parallelized Computation

The annealing call dominates the cost of the forward pass, and the calls for different samples are independent. We exploit this by splitting the batch across a pool of worker threads, each holding its own sampler instance and solving a disjoint subset of the inputs. This keeps the wall-clock cost of embedding an Ising solver in a network manageable on multicore hardware.

B. THE ISING NETWORK

A single module exposes one coupling matrix and one ground-state energy. To add capacity, we place M Ising modules in parallel and combine their outputs with a trainable

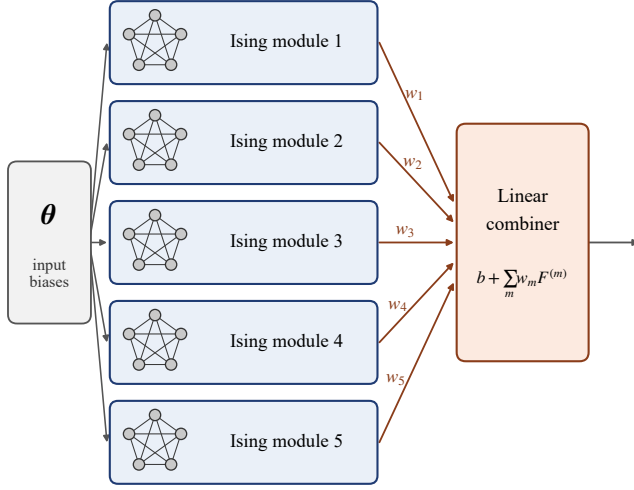


FIGURE 2. Schematic of the Ising network. The input biases θ are shared across $M = 5$ parallel Ising modules, each one an Ising model whose spins and pairwise couplings $\Gamma^{(m)}$ are drawn as a small fully connected graph. Every module returns a scalar $F^{(m)} = \lambda^{(m)} E_0 + \varepsilon^{(m)}$, and the trainable linear combiner aggregates these outputs as $b + \sum_m w_m F^{(m)}$ to produce the network output $F(\theta)$.

linear readout; the result is the Ising network, a single-layer architecture in which each unit is an independent Ising model.

Given an input vector θ , the m -th module produces

$$F^{(m)}(\theta) = \lambda^{(m)} E_0(\theta, \Gamma^{(m)}) + \varepsilon^{(m)}, \quad (13)$$

with its own couplings $\Gamma^{(m)}$ and its own scale and offset $\lambda^{(m)}, \varepsilon^{(m)}$. The M outputs are aggregated by a linear readout,

$$F(\theta) = b + \sum_{m=1}^M w_m F^{(m)}(\theta), \quad (14)$$

with trainable weights w_m and bias b .

The construction mirrors a single fully connected layer, except that each neuron is an Ising module: the nonlinearity comes from the discrete ground-state computation inside each unit, and the linear readout combines the resulting energy-based features. Each module therefore exposes a different nonlinear feature of the data, and the readout aggregates these features with weights of arbitrary sign, yielding an affine combination of the module energies. Even though the architecture stays shallow, distinct modules can settle on different coupling patterns, which gives the network more expressive power than a single module.

IV. EMPIRICAL EVALUATION

We compare the single Ising module with the Ising network, $M = 5$ parallel modules aggregated through a linear readout. The evaluation is split in three blocks. The first uses nine UCI benchmarks in their native form, which for most of them is class-imbalanced. The second runs the same pipeline on a

pre-balanced variant of those datasets, removing the class prior. The third uses synthetic d -dimensional XOR problems as a controlled probe of non-linear capacity. The two UCI blocks share the datasets, the training pipeline and the hyperparameters and differ only in data preparation, which separates the effect of class imbalance from that of model capacity; the XOR block then probes the architectural gap between the two models. A final experiment repeats the 2D XOR comparison on a D-Wave quantum processor.

A. METHOD

1) Datasets

We adopt nine binary classification datasets from the UCI Machine Learning Repository [28], spanning feature dimensionalities from 3 to 44 and sample sizes from 100 to 1372. The UCI suite is complemented by a family of synthetic d -dimensional XOR datasets for $d \in \{1, \dots, 6\}$, a controlled stress test for higher-order non-linear interactions. Table 1 summarizes the size and per-class distribution of both groups.

TABLE 1. Summary of the datasets used in the experimental evaluation.

Dataset	# Features	# Samples	Class dist. (0/1)
00_iris_versicolor_virginica	4	100	50 / 50
01_vertebral_column	6	200	100 / 100
02_banknote	4	1372	762 / 610
03_breast_cancer	30	569	357 / 212
04_contraceptive_method	9	962	629 / 333
05_haberman_survival	3	306	225 / 81
06_heart_failure	12	299	203 / 96
07_ionosphere	34	351	126 / 225
08_SPECTF_Heart	44	267	55 / 212
<i>Synthetic XOR datasets</i>			
XOR-1D	1	200	100 / 100
XOR-2D	2	400	200 / 200
XOR-3D	3	800	400 / 400
XOR-4D	4	1600	800 / 800
XOR-5D	5	3200	1600 / 1600
XOR-6D	6	6400	3200 / 3200

2) Evaluation Protocol

UCI performance is assessed under two complementary forms that share the same model, training pipeline and hyperparameters, and differ only in how the data fed to the cross-validation procedure is constructed.

a: Original (imbalanced) datasets

Each UCI dataset is used in its native form, with the class distribution reported in Table 1. Stratified 5-fold cross-validation preserves the original class ratios, so the training and test splits keep the native distribution. This form measures how the two architectures behave when no class-balancing intervention is applied.

b: Pre-balanced fixed folds (random subsampling)

For each UCI dataset a class-balanced variant is prebuilt by random undersampling of the majority class, and the resulting CSV is stored on disk. The same stratified 5-fold splitter is then applied to the already balanced files, so every fold contains an equal number of class 0 and class 1 samples

in both the training and the test split. Because the folds are produced from a fixed file and the random seed is fixed (Table 2), the splits are deterministic and shared across the two architectures.

c: Shared protocol

In both forms, feature standardization is fit on the training fold only and applied to the test fold (preventing information leakage), the random seed is held fixed across runs, and five metrics are collected and averaged across folds: classification accuracy, precision, recall, F_1 score, and area under the ROC curve (AUC) [29]. Accuracy alone is not informative on the imbalanced form, where a classifier that always predicts the majority label already scores the class prior: the F_1 score, the harmonic mean of precision and recall on the positive class, drops as soon as one class is ignored, and the AUC measures the ranking quality of the predicted scores independently of the decision threshold and of the class ratio. For the synthetic XOR problems we use a single stratified 80/20 train/test split with the same training pipeline as the UCI experiments.

3) Training and Inference Procedure

The training phase is implemented in PyTorch. Both architectures are trained with the Adam optimizer and the binary cross-entropy with logits loss (BCEWithLogitsLoss), for a fixed number of epochs with mini-batch gradient descent. At inference time the sigmoid activation is applied to the logits to obtain class probabilities, and a threshold of 0.5 is used for binary classification.

4) Hyperparameter Configuration

A single set of hyperparameters is shared across all UCI and XOR experiments (Table 2). The values were chosen to give stable performance across the considered benchmarks rather than tuned per architecture or per dataset, so that the comparison reflects architectural differences and not per-task hyperparameter gains.

TABLE 2. Hyperparameters used in all experiments. Values are shared between the single Ising module and the Ising network; the combiner learning rate applies to the Ising network only, and the K-Fold cross-validation is used on the UCI benchmarks only.

Hyperparameter	Value
<i>Optimization</i>	
Optimizer	Adam
Loss	BCEWithLogitsLoss
Number of epochs	200
Batch size	32
K-Folds (UCI only)	5
Random seed	42
<i>Per-parameter learning rates</i>	
η_Γ (couplings)	1×10^{-3}
η_λ (energy scale)	5×10^{-3}
η_ε (offset)	1×10^{-2}
η_{comb} (linear combiner)	5×10^{-3}
<i>Architecture</i>	
Annealer size n_{total}	50 spins
Ising modules M (Ising network)	5
Partition input across modules	False (full input replicated)
Hidden-nodes offset ε_h	-0.02
Γ initialization	0.0
λ initialization	1.0 (± 0.1)
ε initialization	1.0 (± 0.1)
# trainable parameters (single Ising module)	1 227
# trainable parameters (Ising network)	6 141
<i>Simulated annealing</i>	
Inverse temperature β range	[1, 10]
Number of reads	1
Number of sweeps	1 000
Sweeps per β step	1

B. RESULTS

1) Native (Imbalanced) UCI Benchmarks

Tables 3 and 4 collect the per-dataset metrics for the single module and the network. Measured by AUC, the network matches or exceeds the single module on every benchmark, with the widest gaps on breast cancer, banknote and heart failure. The difference between the two models lies less in the final accuracy than in how they reach it. On four datasets the single module stops separating the classes and collapses onto a single label. On breast cancer, haberman and heart failure it predicts almost exclusively the negative class, so precision, recall and F_1 drop to or near zero while accuracy merely tracks the share of negative samples; on SPECTF the collapse goes the other way, with the module predicting only the positive class, which gives a recall of one and a precision equal to the positive-class frequency. On haberman and heart failure this single-label behaviour even pushes the module's accuracy above that of the network, even though the classes are not actually separated. The network shows no such collapse on any of these datasets.

Several entries in Table 3 look strong at first glance, but they do not reflect genuine class separation. When a binary classifier defaults to predicting the majority label, its accuracy mechanically matches the class prior; the rows in which precision and recall collapse make this pattern explicit. The balanced form in Section IV-B2 removes this shortcut and provides a cleaner view of what each model actually learns.

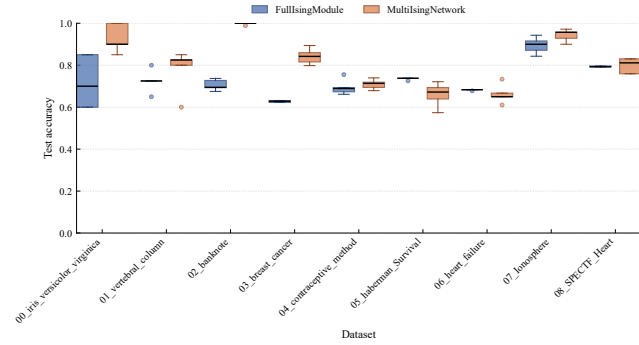
TABLE 3. Single Ising module on the UCI datasets in the imbalanced form. 5-fold CV, mean $\pm$ std.

Dataset	Accuracy	Precision	Recall	F_1	AUC
iris (versicolor/virginica)	0.72 $\pm$ 0.11	0.74 $\pm$ 0.10	0.68 $\pm$ 0.20	0.70 $\pm$ 0.14	0.76 $\pm$ 0.14
vertebral column	0.73 $\pm$ 0.05	0.71 $\pm$ 0.07	0.78 $\pm$ 0.09	0.74 $\pm$ 0.05	0.80 $\pm$ 0.05
banknote	0.71 $\pm$ 0.02	0.68 $\pm$ 0.04	0.67 $\pm$ 0.11	0.66 $\pm$ 0.05	0.72 $\pm$ 0.02
breast cancer	0.63 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.50 $\pm$ 0.06
contraceptive method	0.69 $\pm$ 0.03	0.59 $\pm$ 0.07	0.40 $\pm$ 0.12	0.47 $\pm$ 0.10	0.72 $\pm$ 0.04
haberman	0.74 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.52 $\pm$ 0.07
heart failure	0.68 $\pm$ 0.00	0.30 $\pm$ 0.40	0.02 $\pm$ 0.03	0.04 $\pm$ 0.05	0.42 $\pm$ 0.11
ionosphere	0.89 $\pm$ 0.03	0.89 $\pm$ 0.04	0.96 $\pm$ 0.01	0.92 $\pm$ 0.02	0.96 $\pm$ 0.03
SPECTF	0.79 $\pm$ 0.00	0.79 $\pm$ 0.00	1.00 $\pm$ 0.00	0.89 $\pm$ 0.00	0.71 $\pm$ 0.02

TABLE 4. Ising network on the UCI datasets in the imbalanced form. 5-fold CV, mean $\pm$ std.

Dataset	Accuracy	Precision	Recall	F_1	AUC
iris (versicolor/virginica)	0.93 $\pm$ 0.06	0.92 $\pm$ 0.10	0.96 $\pm$ 0.08	0.93 $\pm$ 0.06	0.93 $\pm$ 0.06
vertebral column	0.78 $\pm$ 0.09	0.80 $\pm$ 0.09	0.75 $\pm$ 0.14	0.77 $\pm$ 0.11	0.88 $\pm$ 0.04
banknote	1.00 $\pm$ 0.00	1.00 $\pm$ 0.01	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00
breast cancer	0.84 $\pm$ 0.03	0.81 $\pm$ 0.08	0.77 $\pm$ 0.08	0.78 $\pm$ 0.04	0.92 $\pm$ 0.03
contraceptive method	0.71 $\pm$ 0.02	0.62 $\pm$ 0.06	0.48 $\pm$ 0.13	0.52 $\pm$ 0.06	0.76 $\pm$ 0.01
haberman	0.66 $\pm$ 0.05	0.26 $\pm$ 0.10	0.15 $\pm$ 0.09	0.18 $\pm$ 0.08	0.58 $\pm$ 0.04
heart failure	0.66 $\pm$ 0.04	0.48 $\pm$ 0.07	0.41 $\pm$ 0.06	0.44 $\pm$ 0.05	0.66 $\pm$ 0.08
ionosphere	0.94 $\pm$ 0.03	0.94 $\pm$ 0.04	0.97 $\pm$ 0.02	0.96 $\pm$ 0.02	0.98 $\pm$ 0.01
SPECTF	0.80 $\pm$ 0.03	0.84 $\pm$ 0.02	0.92 $\pm$ 0.02	0.88 $\pm$ 0.02	0.81 $\pm$ 0.04

Figure 3 summarizes the fold-level dispersion. The network boxes sit at higher accuracy than the single module on most datasets, most clearly on iris, banknote and breast cancer. On haberman, heart failure and SPECTF the single-module box collapses to an almost flat line: the model predicts a near-constant label, so its variance is small and, on haberman and heart failure, this constant prediction even sits above the network.

**FIGURE 3.** Test accuracy on the imbalanced UCI datasets. Each box aggregates the five cross-validation folds for the single Ising module and the Ising network.

2) Pre-balanced UCI Benchmarks

The same comparison is repeated on the balanced variant of the nine UCI sets, everything else held fixed. Per-dataset metrics are reported in Tables 5 and 6. Removing the class-prior shortcut lowers the scores of both models. The drop is larger for the single module, which loses several points of accuracy on average, while the network keeps a clear margin on breast cancer, ionosphere, heart failure and SPECTF. On the sets where the single module previously collapsed to a constant predictor, precision, recall and F_1 are now non-zero: balancing removes the single-label behaviour, and the margins that remain reflect learned class separation rather than the class prior. The exception is haberman, where neither

model exceeds chance. This matches the literature, where haberman is reported as one of the hardest UCI benchmarks: its minority class is dominated by borderline and rare examples lying in the class-overlap region rather than by safe ones, a configuration known to drive classifiers toward the majority baseline regardless of the model [30]. We therefore read the haberman result as a property of the data rather than a shortcoming of either model.

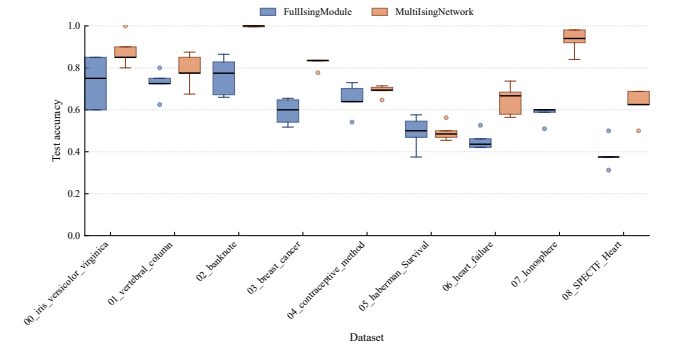
TABLE 5. Single Ising module on the pre-balanced UCI datasets. 5-fold CV on the balanced files, mean $\pm$ std.

Dataset	Accuracy	Precision	Recall	F_1	AUC
iris (versicolor/virginica)	0.73 $\pm$ 0.11	0.74 $\pm$ 0.10	0.70 $\pm$ 0.20	0.71 $\pm$ 0.14	0.77 $\pm$ 0.14
vertebral column	0.73 $\pm$ 0.06	0.71 $\pm$ 0.07	0.78 $\pm$ 0.10	0.74 $\pm$ 0.06	0.79 $\pm$ 0.05
banknote	0.76 $\pm$ 0.08	0.81 $\pm$ 0.11	0.71 $\pm$ 0.17	0.74 $\pm$ 0.11	0.85 $\pm$ 0.04
breast cancer	0.59 $\pm$ 0.06	0.59 $\pm$ 0.05	0.61 $\pm$ 0.12	0.60 $\pm$ 0.07	0.60 $\pm$ 0.07
contraceptive method	0.65 $\pm$ 0.06	0.65 $\pm$ 0.07	0.65 $\pm$ 0.08	0.65 $\pm$ 0.07	0.72 $\pm$ 0.05
haberman	0.49 $\pm$ 0.07	0.52 $\pm$ 0.10	0.57 $\pm$ 0.17	0.52 $\pm$ 0.07	0.49 $\pm$ 0.07
heart failure	0.45 $\pm$ 0.04	0.45 $\pm$ 0.05	0.49 $\pm$ 0.12	0.46 $\pm$ 0.08	0.43 $\pm$ 0.03
ionosphere	0.58 $\pm$ 0.04	0.60 $\pm$ 0.05	0.54 $\pm$ 0.14	0.55 $\pm$ 0.06	0.62 $\pm$ 0.05
SPECTF	0.39 $\pm$ 0.06	0.41 $\pm$ 0.06	0.58 $\pm$ 0.17	0.47 $\pm$ 0.10	0.20 $\pm$ 0.01

TABLE 6. Ising network on the pre-balanced UCI datasets. 5-fold CV on the balanced files, mean $\pm$ std.

Dataset	Accuracy	Precision	Recall	F_1	AUC
iris (versicolor/virginica)	0.88 $\pm$ 0.07	0.89 $\pm$ 0.10	0.88 $\pm$ 0.07	0.88 $\pm$ 0.06	0.94 $\pm$ 0.05
vertebral column	0.79 $\pm$ 0.07	0.80 $\pm$ 0.05	0.78 $\pm$ 0.15	0.78 $\pm$ 0.08	0.87 $\pm$ 0.05
banknote	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00
breast cancer	0.82 $\pm$ 0.02	0.83 $\pm$ 0.03	0.82 $\pm$ 0.05	0.82 $\pm$ 0.02	0.90 $\pm$ 0.02
contraceptive method	0.69 $\pm$ 0.02	0.71 $\pm$ 0.03	0.65 $\pm$ 0.03	0.68 $\pm$ 0.02	0.76 $\pm$ 0.02
haberman	0.49 $\pm$ 0.04	0.53 $\pm$ 0.11	0.34 $\pm$ 0.20	0.37 $\pm$ 0.12	0.51 $\pm$ 0.04
heart failure	0.65 $\pm$ 0.07	0.66 $\pm$ 0.08	0.63 $\pm$ 0.06	0.64 $\pm$ 0.06	0.70 $\pm$ 0.08
ionosphere	0.93 $\pm$ 0.05	0.96 $\pm$ 0.03	0.90 $\pm$ 0.08	0.93 $\pm$ 0.06	0.98 $\pm$ 0.01
SPECTF	0.63 $\pm$ 0.07	0.60 $\pm$ 0.06	0.70 $\pm$ 0.23	0.63 $\pm$ 0.12	0.73 $\pm$ 0.14

Figure 4 reports the fold-level distribution on the balanced folds. With the class prior removed, the single-module boxes drop on the datasets where they had previously collapsed to a flat line, while the network keeps higher and more stable accuracy on the harder benchmarks.

**FIGURE 4.** Test accuracy on the pre-balanced UCI datasets. Each box aggregates the five cross-validation folds for the single Ising module and the Ising network.

3) Synthetic XOR Datasets

The last block tests the two architectures on the synthetic d -dimensional XOR family with $d \in \{1, \dots, 6\}$.

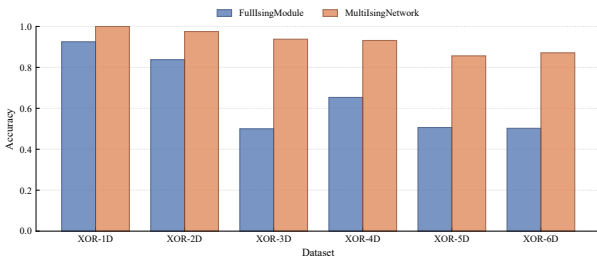
TABLE 7. Performance metrics for the single Ising module on the synthetic d -dimensional XOR problem, for $d \in \{1, \dots, 6\}$ (single stratified train/test split).

Dataset	Accuracy	Precision	Recall	AUC
XOR-1D	0.93	0.90	0.95	0.99
XOR-2D	0.84	0.83	0.85	0.92
XOR-3D	0.50	0.50	0.35	0.58
XOR-4D	0.65	0.65	0.68	0.68
XOR-5D	0.51	0.50	0.84	0.53
XOR-6D	0.50	0.53	0.05	0.58

TABLE 8. Performance metrics for the Ising network on the synthetic d -dimensional XOR problem, for $d \in \{1, \dots, 6\}$ (single stratified train/test split).

Dataset	Accuracy	Precision	Recall	AUC
XOR-1D	1.00	1.00	1.00	1.00
XOR-2D	0.98	0.95	1.00	1.00
XOR-3D	0.94	0.94	0.94	0.98
XOR-4D	0.93	0.94	0.93	0.99
XOR-5D	0.86	0.85	0.87	0.93
XOR-6D	0.87	0.84	0.93	0.96

The two models separate clearly on this family (Fig. 5). The single module already struggles at two dimensions and decays toward chance as the dimension grows, which suggests that one energy minimization cannot represent the parity structure on its own. A finer pattern is visible within the single module: beyond one dimension, the odd-dimensional instances are harder than their even-dimensional neighbours. The 3D and 5D problems sit essentially at chance (accuracy 0.50 and 0.51, AUC 0.58 and 0.53), whereas 2D and 4D stay above it (0.84 and 0.65); the 6D case is the one exception, collapsing to chance as well. A full account of this even/odd gap would require a dedicated analysis of which parity functions a single quadratic Ising energy can express, which we leave to future work. The network, in contrast, resolves the task regardless of parity: it stays well above chance up to six dimensions, because its parallel modules learn complementary coupling patterns whose linear combination reconstructs the higher-order interactions.

**FIGURE 5.** Mean test accuracy on the synthetic XOR problem as a function of the input dimension, for the single Ising module and the Ising network.

a: Simulated vs Quantum Annealing

All experiments reported so far solve the energy minimization with simulated annealing. As a final test on the 2D instance, we run the comparison between the single Ising module and a three-module Ising network ($M = 3$) on two back-

ends: the simulated annealer used above and a D-Wave quantum processing unit (QPU), the Advantage_system4.1 processor. QPU resources are only available on a limited basis, so the experiment replaces the shared configuration of Table 2 with the reduced configuration of Table 9, applied identically to both backends. For the same reason the sampling budgets of the two backends are not matched: the simulated annealer keeps the annealing schedule of Table 2 (inverse temperature range $[1, 10]$, 1000 sweeps per read), while the QPU performs a single anneal-readout cycle per energy evaluation (number of reads equal to one), the minimum possible QPU time per call, so both the forward energy and the gradient are computed from the single spin configuration returned by the hardware. Each backend is run once.

TABLE 9. Reduced hyperparameter configuration used for the simulated vs. quantum annealing comparison. Shared between the single Ising module and the three-module Ising network.

Hyperparameter	Value
Dataset	2D-XOR, 80 points (40 per class)
Train/test split	80/20
Number of epochs	25
Batch size	16
Random seed	42
Annealer size n_{total}	8
Ising modules M	3
η_{Γ}	0.01
η_{λ}	0.02
η_{ϵ}	0.03
η_{comb}	0.02

The purpose of the comparison is twofold: to check whether the simulated and the quantum backend lead to comparable outcomes, and to probe how much the trained performance depends on the quality and completeness of the annealer output, since the module treats the returned configuration as the ground state both in the forward energy and in the gradient. Table 10 reports the test metrics of both models under both backends.

TABLE 10. Test metrics on the 2D XOR problem with the simulated and the quantum annealing backend.

Model	Accuracy	Precision	Recall	F_1	AUC
<i>Simulated annealing</i>					
Single Ising module	0.750	0.750	0.750	0.750	0.875
Ising network ($M = 3$)	0.938	1.000	0.875	0.933	0.984
<i>Quantum annealing</i>					
Single Ising module	0.750	0.750	0.750	0.750	0.875
Ising network ($M = 3$)	0.938	1.000	0.875	0.933	0.984

The two backends give the same picture. Both models reach the same test metrics under simulated and quantum annealing, and the accuracy gap between network and module ($+0.19$) is reproduced on the hardware. A single read per energy evaluation is enough for the QPU trained network to match the result obtained with the 1000 sweep simulated schedule, which suggests that training does not strictly depend on the annealer returning an optimal or exhaustive solution on this task. These observations rest on one run per backend and a 16 point test split, so we take the quantum

result as a reference point, showing that the module and the network train end to end on quantum hardware, rather than as a statistical claim of equivalence between the two backends.

V. DISCUSSION

The experiments give a consistent picture. Across the three blocks the Ising network matches or improves on the single module, and the gap widens where the task is harder. On the native UCI benchmarks the single module often reaches a high accuracy by collapsing onto the majority class, which the precision, recall and F_1 columns expose at once. The network avoids this shortcut on the same datasets: the additional units and the linear readout give it enough capacity to separate the classes rather than exploit their prior.

The balanced version confirms this reading. Once the class prior is removed, the inflated accuracies of the single module disappear and its scores fall to or below chance on the hardest sets, while the network keeps a clear margin on breast cancer, ionosphere, heart failure and SPECTF. The single module is thus more sensitive to class imbalance, and the parallel composition is what restores a usable decision boundary. Haberman is the exception: neither model exceeds chance on it, a result we attribute to the data rather than to the method, for the reasons discussed in Section IV-B2.

The synthetic XOR family isolates the same effect under controlled non-linearity. The single module already struggles in two dimensions and degrades toward chance as the dimension grows, which suggests that a single energy minimization cannot represent the parity structure on its own; the odd-dimensional instances are the hardest, sitting at chance in three and five dimensions. The network stays well above chance up to six dimensions, because the parallel modules learn complementary coupling patterns whose linear combination reconstructs the higher-order interactions. Since all runs share a single hyperparameter set, this is the clearest evidence that the gain comes from the architecture rather than from per-dataset tuning. The reduced 2D XOR run on quantum hardware adds a practical data point: with a single read per energy evaluation, both models trained on the QPU match their simulated counterparts on this task.

A practical remark concerns cost. The dominant expense of the model is the annealing call. Since each module owns its own spins and parameters $\Gamma^{(m)}$, $\lambda^{(m)}$, $\varepsilon^{(m)}$, the modules are independent Ising problems. Several of them can in principle be compiled onto a single physical annealer at once, packing each problem into a different region of the qubit graph. Because the energies act on disjoint variable sets the joint minimization factorizes,

$$\min_{\mathbf{z}} \sum_m E(\boldsymbol{\theta}, \Gamma^{(m)}, z^{(m)}) = \sum_m E_0^{(m)}, \quad (15)$$

that is, the minimizer of the sum is the set of the individual module minimizers. A single anneal therefore returns the ground-state energy and the spin configuration of every module at once. This would amortize the access latency of

a quantum processor over many modules and make wider or deeper Ising networks practical on real hardware.

VI. CONCLUSION

We presented a differentiable Ising energy module that integrates the parametric Ising model of Schmid *et al.* [27] into PyTorch, together with the Ising network, a shallow architecture that composes several such modules through a trainable linear readout. The module carries the analytic coupling gradient into automatic differentiation and behaves as a standard layer, so it can be trained end-to-end with ordinary gradient-based tooling.

The empirical study supports two conclusions. The single module is expressive but fragile under class imbalance, where it tends to collapse onto one class, whereas the network removes this behaviour, improves the metrics and recovers a meaningful decision boundary on most datasets. On the synthetic XOR problems the network captures non-linear interactions up to six dimensions while the single module fails beyond the simplest cases. A reduced 2D XOR experiment on a D-Wave QPU reproduces the simulated results with a single read per energy evaluation, showing that both models also train end-to-end on quantum hardware.

The value of the approach is that an Ising solver becomes a reusable node that drops into a hybrid architecture and trains within the existing PyTorch ecosystem, optimizers, loss functions and learning-rate schedulers included. Placed inside such a network, the module shows the gains expected from added capacity. The study has clear limits: the architecture is shallow, performance rests on a single shared hyperparameter set rather than per-task tuning, and the benchmark suites were run with a simulated annealer, with quantum hardware probed only on a reduced 2D XOR configuration. The shared hyperparameter set is a limit in that the reported scores are likely below the maximum potential of the method, but it also admits a positive reading: the gains of the Ising network hold without any per-task tuning, which speaks for the robustness of the architecture. Future work will explore deeper architectures, the compilation of multiple modules onto a single physical annealer, and larger-scale experiments on quantum annealing hardware.

ACKNOWLEDGMENT

M.M. carried out a substantial part of this work during his Bachelor's studies at the University of Trento. E.T. was supported by the MUR National Recovery and Resilience Plan (PNRR) M4C1I4.1, funded by the European Union under NextGenerationEU. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or The European Research Executive Agency. Neither the European Union nor the granting authority can be held responsible for them.

CODE AND DATA AVAILABILITY

The code for this work is available as a git repository at: <https://github.com/Mattew1717/QuantumAnnealerModule>.

REFERENCES

- [1] E. Ising, "Beitrag zur Theorie des Ferromagnetismus," *Zeitschrift für Physik*, vol. 31, no. 1, pp. 253–258, 1925.
- [2] B. M. McCoy and T. T. Wu, *The two-dimensional Ising model*. Courier Corporation, 2014.
- [3] D. Sherrington and S. Kirkpatrick, "Solvable model of a spin-glass," *Physical Review Letters*, vol. 35, pp. 1792–1796, 1975.
- [4] A. Lucas, "Ising formulations of many NP problems," *Frontiers in Physics*, vol. 2, p. 5, 2014.
- [5] F. Barahona, "On the computational complexity of Ising spin glass models," *Journal of Physics A: Mathematical and General*, vol. 15, no. 10, pp. 3241–3253, 1982.
- [6] K. P. Kalinin and N. G. Berloff, "Computational complexity continuum within Ising formulation of NP problems," *Communications Physics*, vol. 5, no. 1, p. 20, 2022.
- [7] N. Mohseni, P. L. McMahon, and T. Byrnes, "Ising machines as hardware solvers of combinatorial optimization problems," *Nature Reviews Physics*, vol. 4, pp. 363–379, 2022.
- [8] Y. Gao, G. Chen, L. Qi, W. Fu, Z. Yuan, and A. J. Danner, "Photonic Ising machines for combinatorial optimization problems," *Applied Physics Reviews*, vol. 11, no. 4, 2024.
- [9] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [10] T. Kadowaki and H. Nishimori, "Quantum annealing in the transverse Ising model," *Physical Review E*, vol. 58, no. 5, pp. 5355–5363, 1998.
- [11] E. Farhi, J. Goldstone, S. Gutmann, and M. Sipser, "Quantum computation by adiabatic evolution," arXiv preprint quant-ph/0001106, 2000.
- [12] A. Das and B. K. Chakrabarti, "Colloquium: Quantum annealing and analog quantum computation," *Reviews of Modern Physics*, vol. 80, pp. 1061–1081, 2008.
- [13] C. C. McGeoch, *Adiabatic quantum computation and quantum annealing: Theory and practice*. Springer Nature, 2022.
- [14] D-Wave Systems Inc., "D-Wave Ocean software documentation," <https://docs.ocean.dwavesys.com/en/stable/>, accessed: 2025.
- [15] L. Alzubaidi, J. Zhang, A. J. Humaidi, A. Al-Dujaili, Y. Duan, O. Al-Shamma, J. Santamaría, M. A. Fadhel, M. Al-Amidie, and L. Farhan, "Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions," *Journal of Big Data*, vol. 8, p. 53, 2021.
- [16] M. Benedetti, E. Lloyd, S. Sack, and M. Fiorentini, "Parameterized quantum circuits as machine learning models," *Quantum Science and Technology*, vol. 4, no. 4, p. 043001, 2019.
- [17] M. Schuld, I. Sinayskiy, and F. Petruccione, "An introduction to quantum machine learning," *Contemporary Physics*, vol. 56, no. 2, pp. 172–185, 2015.
- [18] V. Dunjko and H. J. Briegel, "Machine learning & artificial intelligence in the quantum domain: a review of recent progress," *Reports on Progress in Physics*, vol. 81, no. 7, p. 074001, 2018.
- [19] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii, "Quantum circuit learning," *Physical Review A*, vol. 98, no. 3, p. 032309, 2018.
- [20] D. Wierichs, J. Izaac, C. Wang, and C. Y.-Y. Lin, "General parameter-shift rules for quantum gradients," *Quantum*, vol. 6, p. 677, 2022.
- [21] D. Pastorello, *Concise guide to quantum machine learning*. Springer, 2023.
- [22] F. Tacchino, C. Macchiavello, D. Gerace, and D. Bajoni, "An artificial neuron implemented on an actual quantum processor," *npj Quantum Information*, vol. 5, p. 26, 2019.
- [23] A. Mari, T. R. Bromley, J. Izaac, M. Schuld, and N. Killoran, "Transfer learning in hybrid classical-quantum neural networks," *Quantum*, vol. 4, p. 340, 2020.
- [24] D. Willsch, M. Willsch, H. De Raedt, and K. Michielsen, "Support vector machines on the D-Wave quantum annealer," *Computer Physics Communications*, vol. 248, p. 107006, 2020.
- [25] P. Date, D. Arthur, and L. Pusey-Nazzaro, "QUBO formulations for training machine learning models," *Scientific Reports*, vol. 11, no. 1, p. 10029, 2021.
- [26] M. H. Amin, E. Andriyash, J. Rolfe, B. Kulchitsky, and R. Melko, "Quantum Boltzmann machine," *Physical Review X*, vol. 8, no. 2, p. 021050, 2018.
- [27] L. Schmid, E. Zardini, and D. Pastorello, "A general learning scheme for classical and quantum Ising machines," *SciPost Phys. Core*, vol. 7, p. 013, 2024. [Online]. Available: <https://scipost.org/10.21468/SciPostPhysCore.7.1.013>
- [28] D. Dua and C. Graff, "UCI machine learning repository," <http://archive.ics.uci.edu/ml>, 2019.
- [29] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [30] K. Napierala and J. Stefanowski, "Types of minority class examples and their influence on learning classifiers from imbalanced data," *Journal of Intelligent Information Systems*, vol. 46, no. 3, pp. 563–597, 2016.

...